

Reporte de actividades

Busque dos cursos en Udemy de angular y sql server y saque todos los temas que vienes e hice un alista, pero de .net sincerament no me convencieron los cursos que habian, entonces mejor le pregunte a la ia si me podía hacer una lista de lo que me faltaba aprender y me dio la lista

The screenshot shows the Udemy course page for 'Angular: De cero a experto'. The course has a rating of 4.8 stars from 19,977 reviews and 70,913 students. The course content is organized into sections: 'Introducción' (5 classes, 22 min), 'Conceptos generales - TypeScript y Angular' (3 classes, 14 min), 'Bases de TypeScript - Recomendado' (21 classes, 2 h 27 min), 'Angular' (15 classes, 1 h 38 min), and 'Expandir bases' (20 classes, 2 h 3 min). The right sidebar shows the purchase options: individual course for 199 MX\$ (29% discount from 279 MX\$), with a 30-day refund guarantee and lifetime access. A coupon code 'CP260518SUMXLD' is applied, resulting in a price of 165,00 MX\$.

Angular: De cero a experto
Lo más vendido | Mejor valorados | 4.8 ★ (19,977 valoraciones) | 70,913 estudiantes

Contenido del curso
24 secciones • 345 clases • 33 h 32 m de duración total [Ampliar todas las secciones](#)

- Introducción** (5 clases • 22 min)
 - Introducción (3:34) [Vista previa](#)
 - ¿Cómo funciona el curso? (4:31) [Vista previa](#)
 - Instalaciones recomendadas (11:25)
 - Hoja de atajos - Recomendaciones (2:09)
 - ¡Únete a Nuestra Comunidad de DevTalles en Discord!
- Conceptos generales - TypeScript y Angular** (3 clases • 14 min)
- Bases de TypeScript - Recomendado** (21 clases • 2 h 27 min)
- Angular** (15 clases • 1 h 38 min)
- Expandir bases** (20 clases • 2 h 3 min)

Compra de cursos individual
199 MX\$ ~~279 MX\$~~
29 % de descuento
Quedan 22 horas a este precio.
Garantía de reembolso de 30 días
Acceso de por vida
[Añadir a la cesta](#)
[Comprar ahora](#)

Suscríbete y ahorra
Desde 165,00 MXN

[Aplicar cupón](#)
Introducir el cupón
CP260518SUMXLD [¡Se ha aplicado!](#)

The screenshot shows the Udemy course page for 'SQL Server: Programación Avanzada 2025 [OFICIAL]'. The course has a rating of 4.8 stars from 1,956 reviews and 7,375 students. The course content is organized into sections: 'Introducción a curso' (3 classes, 3 min), 'Instalación y Setup del software necesario' (10 classes, 30 min), and 'Expandir bases' (20 classes, 2 h 3 min). The right sidebar shows the purchase options: individual course for 139 MX\$ (30% discount from 199 MX\$), with a 30-day refund guarantee and lifetime access. A coupon code 'CP260518SUMXLD' is applied, resulting in a price of 165,00 MX\$.

SQL Server: Programación Avanzada 2025 [OFICIAL]
4.8 ★ (1,956 valoraciones) | 7,375 estudiantes

Contenido del curso
13 secciones • 76 clases • 5 h 38 m de duración total [Ampliar todas las secciones](#)

- Introducción a curso** (3 clases • 3 min)
 - Velocidad de reproducción y Marcadores (2:08) [Vista previa](#)
 - Grupo exclusivo de alumnos (0:45)
 - Suscríbete al Grupo Exclusivo de Alumnos (0:09)
- Instalación y Setup del software necesario** (10 clases • 30 min)
 - Introducción (1:12) [Vista previa](#)
 - Software necesario (0:19)
 - Instalación del Motor SQL Server Developer en Windows (12:07)
 - Comprobando el servicio (1:52)
 - Instalación de SQL Server Management Studio (3:00)
 - Conectando con el motor SQL (3:05)
- Expandir bases** (20 clases • 2 h 3 min)

Compra de cursos individual
139 MX\$ ~~199 MX\$~~
30 % de descuento
Quedan 22 horas a este precio.
Garantía de reembolso de 30 días
Acceso de por vida
[Añadir a la cesta](#)
[Comprar ahora](#)

Suscríbete y ahorra
Desde 165,00 MXN

[Aplicar cupón](#)
Introducir el cupón
CP260518SUMXLD [¡Se ha aplicado!](#)

SQL Server

1. Campos `IDENTITY`
2. Archivos `MDF` y `LDF`
3. Tipos de datos de usuario
4. `CHECK CONSTRAINT`
5. Valores `DEFAULT`
6. Campos de estado
7. Soft delete
8. Tablas resumen
9. `INSERT SELECT`
10. `UPDATE` con `INNER JOIN`
11. `DELETE` con `INNER JOIN`
12. `SELECT INTO`
13. `CASE WHEN`
14. `UNION`
15. `UNION ALL`
16. `TOP`
17. `PIVOT`
18. `UNPIVOT`
19. `SET IDENTITY\_INSERT`
20. `WITH (NOLOCK)`
21. `DATALENGTH`
22. `CAST`
23. `CONVERT`
24. `PARSE`
25. `TRY\_CAST`
26. `TRY\_CONVERT`
27. `TRY\_PARSE`
28. `FORMAT`
29. `STRING\_AGG`
30. `IIF`
31. `CHOOSE`
32. `DBCC CHECKDB`
33. `DBCC CHECKALLOC`
34. `DBCC SHRINKDATABASE`
35. `DBCC SQLPERF(LOGSPACE)`
36. `DBCC FREEPROCCACHE`

37. SQL Server Agent Jobs

38. Practicar transacciones con casos reales

39. Practicar `MERGE`

40. Practicar window functions con ejercicios

41. Practicar CTEs recursivos

Angular

1. `HttpClient`
2. `provideHttpClient`
3. `HttpClient.get<T>()`
4. `Observable`
5. `subscribe`
6. `firstValueFrom`
7. Servicios en Angular aplicados a API
8. Inyección de dependencias en Angular
9. Manejo de errores HTTP
10. Estados de carga: loading, error, empty, success
11. `computed`
12. `effect`
13. `readonly signals`
14. `RouterLinkActive`
15. Rutas hijas
16. Layout components
17. Query params
18. Navegación programática con `Router.navigate`
19. Inputs entre componentes
20. Signal inputs
21. Outputs entre componentes
22. Comunicación hijo-padre
23. Componentes reutilizables
24. `ngClass`
25. Clases condicionales
26. Pipes
27. `DecimalPipe`
28. `DatePipe`
29. Formularios template-driven
30. Formularios reactivos

31. Validaciones en formularios
32. Environments
33. Path aliases
34. LocalStorage
35. Guards
36. Interceptors
37. Resources
38. RxResource
39. `viewChild`
40. Infinite scroll
41. Preservar posición del scroll
42. Testing de componentes
43. Angular CLI más profundo

.NET

1. `HttpPost`
2. `HttpPut`
3. `HttpPatch`
4. `HttpDelete`
5. CRUD completo
6. DTOs de entrada
7. `CreateProductDto`
8. `UpdateProductDto`
9. Validaciones con Data Annotations
10. `[Required]`
11. `[MaxLength]`
12. `[Range]`
13. Model validation
14. `BadRequest`
15. `NoContent`
16. `CreatedAtAction`
17. `Conflict`
18. Interfaces para services
19. `IProductService`

20. Repository Pattern
21. Manejo global de errores
22. Middleware
23. `ProblemDetails`
24. `ILogger`
25. Configuración por ambiente
26. Variables de entorno
27. User Secrets
28. CORS
29. Swagger / OpenAPI
30. EF Core Migrations
31. Relaciones en EF Core
32. `Include`
33. LINQ avanzado
34. `GroupBy`
35. `Any`
36. `All`
37. `SingleOrDefault`
38. Tracking vs No Tracking más profundo
39. Consultas raw SQL en EF Core
40. Transacciones con EF Core
41. `CancellationToken`
42. Autenticación JWT
43. Autorización
44. `[Authorize]`
45. Roles
46. Claims
47. Unit testing
48. Integration testing
49. Arquitectura por capas
50. Clean Architecture básica
51. Docker para .NET
52. Health Checks

Algo que me gusto mientras estaba buscando lo de los cursos fue que encontré una pagina para practicar sql , y hice varios ejercicios, algunos no me salieron sinceramente, asi que veía la respuesta y luego recordaba lo que vi, y la verdad me ayudó mucho, al inicio estaba batallando pero los últimos que hice ya los estaba haciendo sin ayuda, y pues me di cuenta también que conceptos aun me falta practicar

The image displays three screenshots of a SQL practice interface, each showing a 'Question Selection' panel and a 'Filters' panel. The interface is designed for practicing SQL queries on a database named 'Hospital'.

Question Selection Panel:

- Question 1:** Show first name, last name, and gender of patients whose gender is 'M'. (EASY, COMPLETED)
- Question 2:** Show first name and last name of patients who does not have allergies. (null) (EASY, COMPLETED)
- Question 3:** Show first name of patients that start with the letter 'C' (EASY, COMPLETED)
- Question 4:** Show first name and last name of patients that weight within the range of 100 to 120 (inclusive) (EASY, COMPLETED)
- Question 5:** Update the patients table for the allergies column. If the patient's allergies is null then replace it with 'NKA' (EASY, COMPLETED)
- Question 6:** Show first name and last name concatenated into one column to show their full name. (EASY, COMPLETED)

Filters Panel:

- Keywords:** ALL, Case, Distinct, Group By, Having, In, Join, Like, Null, Union, Where, Count, Avg, Sum, Max, Min
- Difficulty:** ALL, Easy, Medium, Hard
- Completion:** ALL, Incomplete, Complete
- Database:** Hospital, Northwind

Question Selection Panel (Second Screenshot):

- Question 1:** Show first name, last name, and the full province name of each patient. Example: 'Ontario' instead of 'ON' (EASY, COMPLETED)
- Question 2:** Show how many patients have a birth_date with 2010 as the birth year. (EASY, COMPLETED)
- Question 3:** Show the first_name, last_name, and height of the patient with the greatest height. (EASY, COMPLETED)
- Question 4:** Show all columns for patients who have one of the following patient_ids: 1,45,534,879,1000 (EASY, COMPLETED)
- Question 5:** Show the total number of admissions (EASY, COMPLETED)
- Question 6:** Show all the columns from admissions where the patient was admitted and discharged on the same day. (EASY, COMPLETED)

Question Selection Panel (Third Screenshot):

- Question 1:** Show all the columns from admissions where the patient was admitted and discharged on the same day. (EASY, COMPLETED)
- Question 2:** Show the patient id and the total number of admissions for patient_id 579. (EASY, COMPLETED)
- Question 3:** Based on the cities that our patients live in, show unique cities that are in province_id 'NS'. (EASY, COMPLETED)
- Question 4:** Write a query to find the first_name, last name, and birth date of patients who has height greater than 160 and weight greater than 70 (EASY, COMPLETED)
- Question 5:** Write a query to find list of patients first_name, last_name, and allergies where allergies are not null and are from the city of 'Hamilton' (EASY, COMPLETED)
- Question 6:** Show unique birth years from patients and order them by ascending. (MEDIUM, COMPLETED)

Aquí estan los ejercicios que hice, serán pocos pero ahí lo estaba haciendo con lentitud en lo que asimilaba nuevamente la sintaxis, ya depues iba a empezar con los primeros conceptos de la lista pero ya no me dio chance porque ya me tuve que ir de la uat

```
-- select first_name, last_name, gender
```

```
-- from patients
```

```
-- where gender = 'M'
```

```
--select first_name, last_name, allergies
```

```
--from patients
```

```
--where allergies is null
```

```
--select first_name
```

```
--from patients
```

```
--where first_name like 'c%'
```

```
--select first_name, last_name
```

```
--from patients
```

```
--where weight between 100 and 120
```

```
--where weight >= 100 and weight <= 120
```

```
--update patients
```

```
--set allergies = 'NKA'
```

```
--where allergies IS null
```

```
--select concat( first_name,', ' last_name )as full_name
```

```
--from patients
```

```
--select a.first_name, a.last_name, b.province_name
```

```
--from patients as a
```

```
--inner join province_names as b
```

```
--on a.province_id = b.province_id
```

```
--select count(*) as total
```

```
--from patients
```

```
--where year(birth_date) = 2010
```

```
--SELECT
```

```
-- MAX(height)
```

```
--FROM patients;
```

```
--SELECT
```

```
-- first_name,
```

```
-- last_name,
```

```
-- height
```

```
--FROM patients
```

```
--WHERE height = (
```

```
--  SELECT max(height)
```

```
--  FROM patients
```

```
-- )
```

```
SELECT *
```

```
FROM patients
```

```
WHERE patient_id IN (1, 45, 534, 879, 1000);
```

```
select count(*) as total_admissions  
from admissions
```

```
select * from admissions  
where admission_date = discharge_date
```

```
select patient_id, count(*) as total_admissions  
from admissions  
where patient_id = 579
```

```
select city from patients  
where province_id = 'NS'  
group by city
```

```
SELECT DISTINCT(city) AS unique_cities  
FROM patients
```

```
select first_name, last_name, birth_date  
from patients  
where height >= 160 and weight >= 70
```

```
select first_name, last_name, allergies  
from patients  
where city = 'Hamilton' and allergies is not null
```

```
select first_name  
from patients  
group by first_name  
having count(*) = 1
```

```
select patient_id, first_name  
from patients  
where first_name like 's%s' and len(first_name) >= 6
```